

SMART CAMPUS & PLACEMENT PLATFORM

Project V2 — Domain Expansion & Relational Backend Blueprint

1. What Project V2 means

V2 is the first major functional expansion. V1 proved that the project can expose a clean, validated Student API. V2 turns that isolated CRUD module into a connected Smart Campus & Placement system with real domain relationships and business meaning.

2. V2 target domain

Entity	Purpose	Relationships
Student	Student profile and placement information	Skills, Projects, Applications
Skill	Technology/skill associated with a student	Student; later Job requirements
Project	Projects completed by a student	Student
Company	Organization recruiting candidates	Jobs, recruiters
Job	Open position and requirements	Company, Applications
Application	Student application to a job	Student, Job
Assessment	Skill/readiness evaluation	Student; skills/results
Interview	Recruitment stage information	Application
User / Account	Authentication identity	Roles and domain profile

3. V2 learning sequence

Stage A — JPA relationships

- Understand why relational relationships are required before writing annotations.
- Implement Student ↔ Project using one-to-many / many-to-one.
- Understand @JoinColumn, foreign keys, ownership, and mappedBy.
- Introduce Student ↔ Skill using many-to-many and an explicit join table.
- Learn cascade types and when cascade is dangerous.
- Learn orphanRemoval and lifecycle ownership.
- Understand lazy vs eager loading and why default loading decisions matter.
- Demonstrate the N+1 query problem and learn practical ways to avoid it.

Stage B — Build the domain modules

- Project module: entity → repository → service → DTO → mapper → controller → test.
- Skill module with student association.
- Company module for recruiter organizations.
- Job module with company ownership and job requirements.
- Application module connecting Student and Job.
- Interview module connected to Application.

Stage C — Queries and list APIs

- Derived Spring Data query methods.
- JPQL and custom queries when derived methods are insufficient.

- Filtering and search for jobs, students, and applications where useful.
- Pagination for potentially large result sets.
- Sorting.
- Database indexes based on real query patterns rather than premature optimization.

Stage D — Transactions

- Understand what a transaction guarantees.
- Learn atomicity and rollback using real project scenarios.
- Place transaction boundaries around business operations rather than arbitrary controller methods.
- Use `@Transactional` deliberately for multi-step operations such as application workflows.

4. Example V2 business flow

A student applies for a job: HTTP request → ApplicationRequest → ApplicationService → validate Student and Job → create Application entity → persist within a deliberate transaction → ApplicationResponse.

5. Architecture target

Client → Controller → Request DTO → Service → Mapper → Entity → Repository → Spring Data JPA → Hibernate → PostgreSQL. Responses follow the reverse path through Response DTOs.

As the domain grows, services will coordinate multiple repositories when a business operation crosses entities. Controllers should remain thin and should not contain database or relationship-management logic.

6. Engineering questions V2 must answer

- Who owns each relationship?
- Which side controls the foreign key?
- What happens to child data when a parent is deleted?
- When should related data be loaded lazily?
- How can accidental large object graphs be avoided?
- How do we detect and solve N+1 queries?
- When does a business operation require a transaction?
- How should list endpoints behave when the data becomes large?

7. V2 definition of done

- Students can maintain projects and skills.
- Companies can create jobs.
- Students can apply to jobs.
- Applications represent meaningful recruitment status.
- Interviews represent recruitment stages.
- Relationships are understood, tested, and documented.
- Relevant list APIs support pagination/sorting/filtering.
- Transactions are applied deliberately to multi-step operations.
- API responses continue using DTOs rather than exposing JPA entities.

8. What V2 will NOT do

- Create every planned entity at once.

- Jump prematurely into microservices.
- Optimize without understanding an actual problem.
- Copy a large finished architecture without understanding each piece.

9. Documentation checkpoint

Each meaningful V2 module should receive a progress entry describing: what problem it solves, what was implemented, what changed in the architecture, how it was tested, and what remains. The goal is to preserve the reasoning behind the project, not only the final code.

V2 starts when the next major domain feature is intentionally introduced. Until then, V1.2 remains the current implementation checkpoint.